



MARCA: Mamba Accelerator with ReConfigurable Architecture

Jinhao Li^{1*}, Shan Huang^{1*}, Jiaming Xu¹², Jun Liu¹, Li Ding¹, Ningyi Xu¹, Guohao Dai^{1†}

¹Shanghai Jiao Tong University, ²Infinigence-AI, *Equal contributions

[†]Corresponding author: daiguohao@sjtu.edu.cn

ABSTRACT

State space model (SSM) especially Mamba has demonstrated remarkable capabilities in various domains. Compared to Transformers, Mamba reduces the quadratic computational complexity and achieves a higher algorithm accuracy (e.g., the accuracy of Mamba-2.8b is higher than OPT-6.7b). However, challenges still exist in accelerating Mamba computations. **(1) Incompatibility between element-wise operations and Tensor Core.** Linear operations (matrix multiplications) and element-wise operations are the two dominating operations in Mamba. The time proportion of element-wise operations escalates significantly (e.g., >60% with 2048 input length). These operations do not need reduction, which is not compatible with the existing Tensor Core-based architectures (e.g., 1/16 normalized speed). **(2) Large area overhead for nonlinear function unit.** The optimized nonlinear function unit like exponential unit still occupies >30% of the processing element (PE) area. **(3) Large memory access but limited data sharing for element-wise operations.** Linear and element-wise operations in Mamba exhibit large compute intensity variance (e.g., ~3 orders of magnitude) and large read/write ratio variance (e.g., >3 orders). Due to the limited data sharing in element-wise operations, it is useless to apply the existed methods like tiling to element-wise operations.

In response to these challenges, we propose a Mamba accelerator with reconfigurable architecture, MARCA. Then, we propose three novel approaches in this paper. **(1) Reduction alternative PE array architecture** for both linear and element-wise operations. For linear operations, the reduction tree connected to PE arrays is enabled and executes the reduction operation. For element-wise operations, the reduction tree is disabled and the output bypasses. **(2) Reusable nonlinear function unit** based on the reconfigurable PE. We decompose the exponential function into element-wise operations and a shift operation by a fast biased exponential algorithm, and the activation function (SiLU) into a range detection and element-wise operations by a piecewise approximation algorithm. Thus, the reconfigurable PEs are reused to execute nonlinear functions with negligible accuracy loss. **(3) Intra-operation and inter-operation buffer management strategy.** We propose intra-operation buffer management strategy to maximize input data sharing for linear operations within operations, and inter-operation strategy for element-wise operations between operations. We conduct extensive experiments on Mamba model families with different

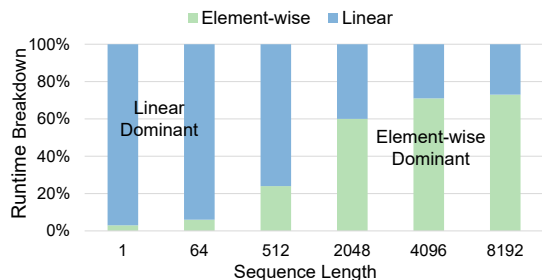


Figure 1: Runtime breakdown with different sequence lengths in Mamba. Element-wise operations contribute a large fraction of the runtime with long sequence length while linear operations are dominant with short length.

sizes. MARCA achieves up to 463.22×/11.66× speedup and up to 9761.42×/242.52× energy efficiency compared to Intel Xeon 8358P CPU and NVIDIA Tesla A100 GPU implementations, respectively.

ACM Reference Format:

Jinhao Li^{1*}, Shan Huang^{1*}, Jiaming Xu¹², Jun Liu¹, Li Ding¹, Ningyi Xu¹, Guohao Dai^{1†}. 2024. MARCA: Mamba Accelerator with ReConfigurable Architecture. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD '24)*, October 27–31, 2024, New York, NY, USA. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3676536.3676798>

1 INTRODUCTION

State space model (SSM) especially Mamba [10] has demonstrated remarkable capabilities in various domains including language, images [7, 35, 48], audio [23], and genomics [27, 41]. Before Mamba, Transformer-based large language models [26, 46] have achieved great success in sequence modeling due to the self-attention mechanism [40]. However, it suffers from handling the quadratic growth of storage and computational complexity as the sequence length increases and struggles to handle sequence with long range dependency. Compared with the pioneers of SSMs [11–13] and Transformer, Mamba achieves higher accuracy in algorithm (e.g., the accuracy of Mamba-2.8b is higher than OPT-6.7b [46]) and more efficient computation in hardware. Therefore, Mamba, as a foundation model [4], has been applied in various domains including vision [34, 48], graphics [2, 16, 41], medical [44], and point cloud [18, 20, 47].

However, there is limited research on optimizing for Mamba processing. Therefore, we profile the processing carefully and identify three main challenges in Mamba computations: **(1) Incompatibility between element-wise operations and Tensor Core.** Linear operations (e.g., matrix multiplication and convolution) and element-wise operations are two dominating operations in Mamba. Tensor Core [5, 24] is a typical domain specific architecture with reduction tree focused on accelerating these linear operations with high the compute intensity (e.g., >1000 FLOPs/Byte). However, as shown in Figure 1, the time proportion of element-wise operations escalates significantly as sequence length increases (e.g., >60% with

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICCAD '24, October 27–31, 2024, New York, NY, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1077-3/24/10

<https://doi.org/10.1145/3676536.3676798>

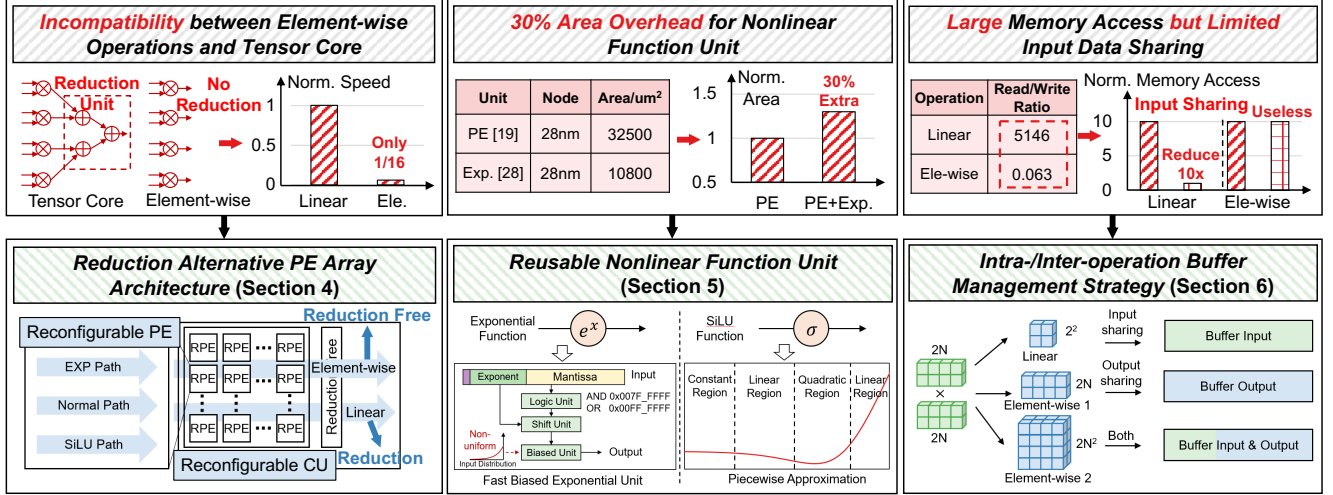


Figure 2: Challenges in Mamba computation: (1) incompatibility between element-wise operations and Tensor Core, (2) 30% area overhead for nonlinear function unit, and (3) large memory access but limited input data sharing for element-wise operations. We propose three novel contributions in MARCA: (1) reduction alternative PE array architecture, (2) reusable nonlinear function unit, and (3) intra-operation and inter-operation buffer management strategy, to solve these challenges.

2048 input length). Because the element-wise operations do not need reduction, applying them on Tensor Core-based architecture should introduce large amount of invalid computations, leading to an extreme inefficiency (e.g., 1/16 normalized speed). **(2) Large area overhead for nonlinear function unit.** Exponential function and Sigmoid Linear Unit (SiLU) function are two nonlinear functions in Mamba. Previous methods [29, 33, 36] often design specific unit to compute them, leading to much more area overheads. As shown in Figure 2 middle bottom, the optimized nonlinear function unit such exponential function still occupy 30% of the PE area [19, 33]. **(3) Large memory access but limited input data sharing for element-wise operations.** Linear and element-wise operations in Mamba exhibit large compute intensity variance (e.g., ~3 orders of magnitude) and large read/write ratio variance (e.g., >3 orders of magnitude). Due to the limited data sharing in element-wise operations, it is useless to apply the existed methods like tiling [17] to element-wise operations.

In response to these challenges, we propose *MARCA*, a Mamba accelerator with reconfigurable architecture, to support fast and energy-efficient Mamba computations. Our contributions are as follows:

(1) Reduction alternative PE array architecture for both linear and element-wise operations. For linear operations, the reduction tree connected to PE arrays is enabled and executes the reduction operation. For element-wise operations, the reduction tree is disabled and bypasses the results.

(2) Reusable nonlinear function unit based on reconfigurable PE arrays. We decompose the exponential function into element-wise operations and a shift operation by a fast biased exponential algorithm. We also decompose the activation function (SiLU) into a range detection and element-wise operations by a piecewise approximation algorithm. Thus, the reconfigurable PEs are fully reused to execute nonlinear functions with negligible accuracy loss.

(3) Intra-operation and inter operation buffer management strategy. We propose intra-operation and inter-operation buffer management strategy for linear and element-wise operations. For

linear operations, the buffer pool is managed as an input buffer to maximize the input data sharing within each operation. For element-wise operations, the buffer pool is managed as an output buffer to maximize the output data sharing between operations.

We implement *MARCA* in *Verilog* and design a cycle-accurate simulator to evaluate *MARCA*. We conduct extensive experiments on Mamba with different model sizes (i.e., Mamba-130M to Mamba-2.8B). As a result, *MARCA* achieves up to 463.22×/11.66× speedup and up to 9761.42×/242.52× energy efficiency compared to Intel Xeon 8358P CPU and NVIDIA Tesla A100 GPU implementations, respectively.

2 BACKGROUND

2.1 State Space Model

The continuous state space model in equation 1 defines a linear mapping from an input signal $x(t) \in \mathbb{R}^M$ (a function of time t) to output signal $y(t) \in \mathbb{R}^M$ through a hidden state $h(t) \in \mathbb{R}^N$:

$$\begin{aligned} h'(t) &= A(t)h(t) + B(t)x(t) \\ y(t) &= C(t)h(t) \end{aligned} \quad (1)$$

where state matrix $A(t) \in \mathbb{R}^{N \times N}$, input matrix $B(t) \in \mathbb{R}^{N \times M}$, output matrix $C(t) \in \mathbb{R}^{M \times N}$, and the change of hidden state $h'(t) \in \mathbb{R}^N$. Classical SSM has dynamic parameters (e.g., A, B, C) that change over time. However, when they are constant the dynamics are invariant through time, which is known as a linear time-invariant (LTI) system [43], and is equivalent to a (continuous) convolution.

Data like words and tokens in the real world is discrete instead of continuous, so equation 1 must be discretized to be applied to an sampled input sequence $x = (x_0, x_1, x_2, \dots)$ instead of continuous function $x(t)$. An additional step size parameter Δ is required that represents the resolution of the input. Conceptually, the inputs $x(n)$ can be viewed as uniformly-spaced samples from an implicit underlying continuous signal $x(t)$, where $x(k) = x(k\Delta)$. The discretization process from continuous time signal processing

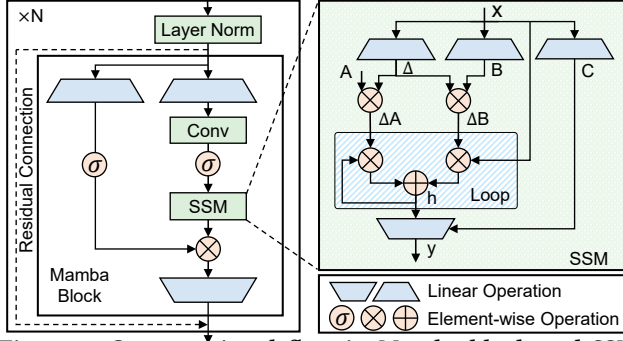


Figure 3: Computational flow in Mamba block and SSM. Mamba model consists of N blocks with residual connection. In SSM, Δ , B and C are generated by input x . Then it performs loops to update hidden state h , and generates output y .

to discrete processing is the fact that the SSM has equivalent forms of solving partial differential equations. To illustrate discretization, the simplest method is to apply Taylor series method [9] which turns the equation 1 into the first-order approximation:

$$\begin{aligned} h_n &= h_{n-1} + \Delta(Ah_{n-1} + Bx_n) \\ &= (I + \Delta A)h_{n-1} + (\Delta B)x_n \\ &= \bar{A}h_{n-1} + \bar{B}x_n \\ y_n &= Ch_n \end{aligned} \quad (2)$$

where the discrete version of parameters has the same shapes as the original continuous version: $A \in \mathbb{R}^{N \times N}$ and $B \in \mathbb{R}^{N \times M}$. The discretized system depends on Δ to generate ΔA and ΔB . Therefore, Δ can be interpreted as parameters that modulates the SSM instead of as a fixed step size.

2.2 Mamba

Mamba [10] introduces selective mechanism into SSM and proposes an implementation of selective state space model layer. Mamba block consists of a layer normalization, several linear projections, a convolution, a SSM block, and a residual connection. In each layer, the input sequence is first processed by a linear projection and then processed by the convolution. Then it is processed by an activation and then processed by the SSM. After SSM, the main branch is multiplied by the collateral branch including a linear projection and an activation (e.g., SiLU [8]) to generate the combined result. After combination, the result is processed by a linear projection. Last, the result is added back to the input through the residual connection [14]. Instead of interleaving Mamba block and Feed-forward block, Mamba simply repeats the Mamba block homogenously.

During SSM processing, the input x undergoes a series of transformations. Firstly, it is subjected to three linear projections, resulting in Δ , B and C . Subsequently, Δ is involved in the Einstein summation operations [1] with A and B separately, generating ΔA and ΔB . These intermediate results ΔA and ΔB are then multiplied with the hidden state and input x by element-wise Einstein summation, respectively. After L (sequence length) times iterations, the hidden state is updated for L times. The outcomes of these operations are then combined through addition, resulting in the updated hidden state. Finally, the updated hidden state undergoes matrix multiplication with C , followed by a linear transformation,

to generate the output. The whole computational flow in Mamba block is illustrated in Figure 3.

3 ARCHITECTURE OVERVIEW

We propose a Mamba accelerator with reconfigurable architecture, MARCA, with reconfigurable computing units (CUs) and processing elements (PEs). MARCA is a reconfigurable architecture whose instructions are all 64-bit, and contains 16 32-bit general-purpose Registers (Regs) and 16 32-bit Constant Registers (CRegs). MARCA consists of four main parts: instruction processing, normalization unit, on-chip buffer, and computing engine, as depicted in Figure. 4 left.

Instruction Processing. The instruction processing consists of two parts: instruction fetch and instruction decode. The instruction fetch unit fetches instructions from global memory and stores them in the instruction buffer. Then, the instruction decode unit reads instructions sequentially from the buffer and decodes them. As shown in Figure 5, the instruction set architecture (ISA) includes linear (LIN), convolution (CONV), normalization (NORM), element-wise multiplication (EWM), element-wise addition (EWA), exponential function (EXP), and SiLU (SILU). And MARCA also provides LOAD and STORE instructions to support moving data between global memory and on-chip buffer. After decoding, the instructions are passed through the configure unit to pass configuration information to the following modules.

Normalization Unit. The layer normalization is an important component that stabilizes range of intermediate values by normalizing layer activation. The normalization unit is responsible for computing the mean and variance of the data. The data is first summarized and accumulated to get the mean and then the variance is calculated. Then, data undergoes a linear unit to obtain the normalized result.

Compute Engine. The compute engine is responsible for linear, convolutional, and element-wise computations. It comprises a control unit and a reconfigurable compute units (RCUs). The control unit receives configuration information from configure unit and fetches data from on-chip buffer to RCU. When the computation is completed, it notifies the instruction processing pipeline to continue decoding and executing the next instruction.

4 REDUCTION ALTERNATIVE PE ARRAY ARCHITECTURE

4.1 Challenge

Linear operation and element-wise operation are two dominating operations in Mamba. Linear operations like matrix multiplications are usually accelerated by Tensor Core, which is a domain specific architecture with the reduction tree. Due to the reduction of partial inner product, the linear operations exhibit extremely high compute intensity (e.g., >1000 FLOPs/Byte). However, as shown in Figure 1, the time proportion of element-wise operations escalates significantly as sequence length increases (e.g., >60% with 2048 input length). Because the element-wise operations do not need reduction, applying them on Tensor Core-based architecture should introduce large amount of invalid computations, leading to an extreme inefficiency (e.g., 1/16 normalized speed). We call it the incompatibility between element-wise operations and Tensor Core.

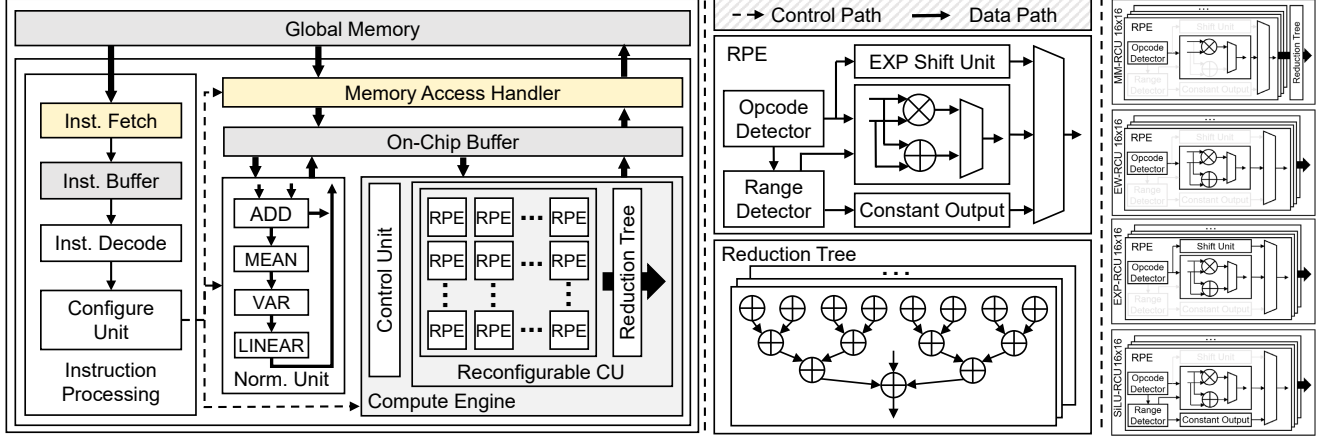


Figure 4: Left: Architecture of MARCA accelerator. MARCA mainly consists of an instruction processing, a normalization unit, an on-chip buffer, and a compute engine. Middle: Architecture of reconfigurable processing element and reduction tree in RCU. Right: Four reconfigurable modes of RCU, MM-RCU, EW-RCU, EXP-RCU, and SiLU-RCU.

4	4	4	4	4	4	4	36
opcode	Reg0	Reg1	Reg2	Reg3	Reg4	Reg5	
LIN/CONV	Out_addr	Out_size	In0_addr	In0_size	In1_addr	In1_size	
4	4	4	4	4	4	4	36
opcode	Reg0	Reg1	Reg2	CReg3	CReg4	CReg5	
EXP/SILU	Out_addr	Out_size	In_addr	Constant0	Constant1	Constant2	
4	4	4	4	4/32	44/16		
opcode	Reg0	Reg1	Reg2	GReg3/Immed			
EWM/EWA	Out_addr	Out_size	In0_addr	In1_addr/Constant			
4	4	4	4	48			
opcode	Reg0	Reg1	Reg2				
NORM	Out_addr	Out_size	In_addr				
4	4	4	4	32			16
opcode	Reg0	Reg1	Reg2	Immed			
LOAD/STORE	Dest_addr	V_size	Src_base	Src_offset			

Figure 5: Instruction set architecture with 16 32-bit general-purpose registers and 16 32-bit constant registers. All instructions are 64-bit.

4.2 Motivation and Insights

Our motivation stems from the inefficiencies observed in element-wise computations on tensor cores. Because the attention operations contain non-linear softmax, previous Transformer accelerators [21, 42] employed independent hardware units to support both attention and linear operations. In contrast, Mamba architecture only consists of linear operations, element-wise operations, and a few activation functions. The only difference of linear of element-wise operations is whether execute reduction or not. Recognizing this simplicity, **our key insight is that by disabling the reduction tree, the Tensor Core-based PE arrays can execute element-wise operations.**

4.3 Approach

Reduction Alternative PE Arrays. The reduction alternative processing element (PE) arrays support configurability. The reduction tree consists of 16 slices (taking 16 for instance) and operates in two modes: reduction mode for linear operations and non-reduction mode for element-wise operations. In reduction mode, the reduction tree is enabled. For a 16-to-1 reduction tree slice, the outputs from 16×1 PE arrays are fed into one reduction tree slice, which employs multi-level additions to compute the sum. In non-reduction mode, the reduction tree is disabled and the outputs from 16×16 PE arrays skip the reduction directly. In addition, the last-level addition in

each slice supports three input to accumulate the partial results for linear operations.

We define that a reconfigurable computing unit (RCU) consists of 16×16 PE arrays and a reduction tree, as shown in Figure 4 left. Each reconfigurable PE (RPE) is configured to support three main computations with three data paths: a shift path for exponential function, a piecewise path for SiLU function, and a normal path for addition or multiplication. The normal path contains a floating-point multiplier, a floating-point adder, and a multiplexer unit to handle element-wise multiplication and addition computations.

Reconfigurable Computing Unit. We provide a detailed explanation of how the RCU operates for the four specific computations in Mamba, as shown in Figure 4 right.

MM-RCU. The RCU is configured as a matrix multiplication mode (MM-RCU) to support linear operations. The reduction tree in RCU and the floating-point multiplier units in RPE are enabled. Therefore, for a matrix multiplication operation of two matrices of size 16×16, the results calculated by the 16×16 array of multipliers are then passed through the reduction tree to produce 16 final results. Then, this process is repeated totally 16 times to obtain the complete result of the 16×16 matrix multiplication. To support the accumulation of partial sums, an additional adder is added at the final level of the reduction tree.

EW-RCU. When RCU is configured as element-wise mode (EW-RCU), the reduction tree is disabled, while the P2D buffer is enabled, and the floating-point multiplier or adder units in RPE are activated. For an element-wise multiplication operation of two 16×16 matrices, the results calculated by the 16×16 array of RPEs maintains the same dimensions of 16×16 and are output to the buffer in parallel.

EXP-RCU. When RCU is configured as exponential mode (EXP-RCU), the reduction tree is disabled. The floating-point multiplier, adder units, and exponential shift unit in RPE are enabled. Therefore, for a 16×16 matrix performing exponential function operation, the RCU first executes element-wise multiplication by using the multipliers, then executes element-wise addition by using the adders. Afterward, the exponential shift unit performs a logic operation, a shift operation and a biased operation to obtain the final output, as shown in Figure 2 right bottom.

SiLU-RCU. When RCU is configured as SiLU mode (SiLU-RCU), the floating-point multiplier or adder, the range detector, and the constant output unit in RPE are enabled while the reduction tree is disabled. For a 16×16 input matrix, each input is first distinguished by range, and then depending on the difference of range, each input is processed by employing either the constant output or normal element-wise computations. The SiLU-RCU decomposes the SiLU operation into 0, 2, or 4 instances of element-wise operations according to equation 3.

5 REUSABLE NONLINEAR FUNCTION UNIT

5.1 Challenge

Exponential function and Sigmoid Linear Unit (SiLU) function are two nonlinear functions in Mamba. Previous methods [29, 33, 36] often design specific unit based on lookup-table or Taylor series approximation to optimize these nonlinear computations. However, the optimized nonlinear function unit such exponential function still occupy 30% of the PE area [19, 33], leading to much more area overheads.

5.2 Motivation and Insights

A common approach is to utilize approximation functions to approximate exponential operations, thereby degrading exponential computations to quadratic or even linear operations. Using linear approximation methods results in large precision loss ($>4\%$ for Mamba-2.8b) while employing higher-order polynomial approximation leads to increased computational overhead. We profile the range of these nonlinear functions. The input for an exponential function is mostly between -7 and 0 , especially for values slightly less than 0 , while the input range of the SiLU function is from -5 to 4 . On one hand, we can only approximate nonlinear functions only in these range to concentrate on preventing accuracy loss. On the other hand, exponential function and SiLU are similar with element-wise operations except for scaling for each value. Therefore, to avoid increasing area overhead and prevent accuracy loss, our key insight is that **only by approximating these nonlinear functions in a small range, we can decompose them into several element-wise operations to reuse the reduction alternative PE array architecture in Section 4.**

5.3 Approach

Fast Biased Exponential Algorithm. Given the peculiar distribution observed in the inputs of the exponential function, namely the outer product of Δ and A , we leverage a set of data points $x = \frac{-7}{n}, n = 1, 2, \dots, 200$ where the density increases as they approach zero to evaluate the deviation of the approximate calculation from the origin exponential value. Consequently, we modified the fast exp algorithm [38] to accommodate specific data ranges and appended a bias at the end to enhance precision and consists of three main steps:

- (1) The input x is linearly transformed into x' .
- (2) x' is multiplied by 2^{23} then cast to an unsigned integer.
- (3) View the x as float-point number and add the bias c .

The fast exp algorithm aims to put $\frac{x}{\ln 2}$ into exponential bit of e^x so that $e^x = 2^{\left(\frac{x}{\ln 2}\right)}$. Float data does not have a direct intuitive

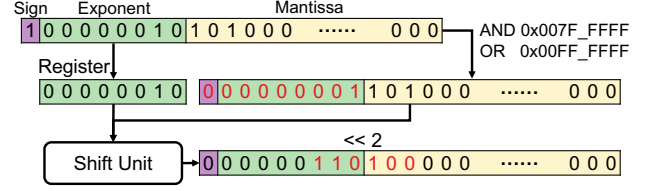


Figure 6: Hardware implementation of exponential shift unit for our fast exponential approximation.

representation like unsigned integers, hence cannot be directly used as the exponent bits for e^x ; therefore, conversion to uint is necessary. Additionally, a bias is required to compensate for conversion loss. The exponent of a float is typically subtracted by 127 during actual computations so we also need consider that. Now we can determine the coefficient $a = \frac{1}{\ln(2)}$ and the term $b = 127 + \text{bias}$, while c serves as the final bias, effectively reducing the average loss caused by specific data distributions.

Piecewise SiLU Algorithm. The computational formula for the SiLU function is $\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1+e^{-x}}$ which involves not only exponentiation but also division operations. Though employing the fast exp algorithm for approximation yields excellent precision, it introduces overhead due to the need for dividers, consequently impacting performance. Our preliminary profiling results indicate that the inputs to the SiLU function are mostly concentrated in the range $[-5, 4]$. Therefore, we propose segmenting the SiLU function within this interval for approximation. Increasing the number of segments enhances precision but also introduces more conditional branches, thus impacting performance. Consequently, we strike a balance between precision and performance and utilize a 4-segment piecewise function to approximate the SiLU function which is expressed by equation 3: Within each interval, further piece-wise approximation is performed based on the distribution of the data.

$$f(x) = \begin{cases} -0.0135, & \text{if } x < -5 \\ -0.06244x - 0.3457, & \text{if } -5 \leq x < -1.5 \\ 0.232(x + 1.181)^2 - 0.275, & \text{if } -1.5 \leq x \leq 0.75 \\ 1.05x - 0.2781, & \text{if } x > 0.75 \end{cases} \quad (3)$$

Reusable Nonlinear Function Unit. Based on two algorithms for nonlinear functions, we decompose the exponential function into element-wise operations and an shift operation, and SiLU into a range detection and element-wise operations. Therefore, we only add a few logics and reuse PE arrays to support nonlinear functions.

As Exponential Function Unit. The original computation process of fast exp is overly complex, particularly the float-to-uint conversion unit incurring significant overhead, we have devised a dedicated conversion unit shown in Figure 6 for fast exp. Within this unit, the input x undergoes floating-point linear computation, resulting in x' . Subsequently, we extract the 8 exponent bits of x' , directly representing the required shift length—positive for left shifts and negative for right shifts. Next, employing logical operations, we convert the original floating-point number into the actual representation of the mantissa. Finally, the result is fed into a shift unit and adjusted through a bias unit to yield the output. Due to the ability to simplify the series of linear transformations applied to the

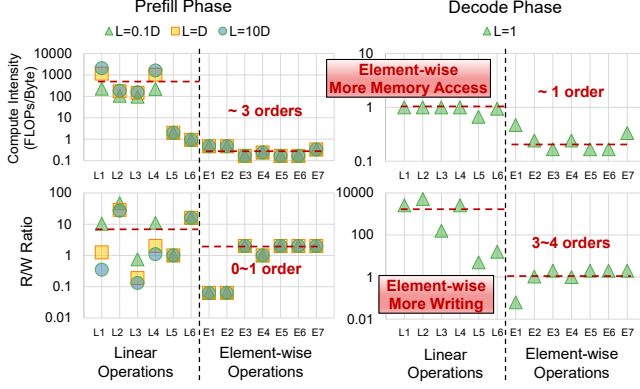


Figure 7: The difference of compute intensity and read/write ratio with different sequence length input in Mamba. Compared with linear operations, element-wise operations need more memory access and more memory writing.

input into a single linear transformation, the actual computation only requires 4 cycles.

As SiLU Unit. The SiLU unit introduces a range detector in the PE unit, responsible for determining the input’s interval and executing the corresponding computation. Due to the most complex operation being quadratic, the actual computational overhead is minimal.

6 INTRA-/INTER-OPERATION BUFFER MANAGEMENT STRATEGY

6.1 Challenge

Linear and element-wise operations are two dominant operations in Mamba processing. The feature of memory access for these two kinds shows three typical paradigms, that is, reading $2 \times 2N$ data and writing $2 \times 2, 2N$, and $2N^2$ corresponding to linear projection (Linear), element-wise addition or multiplication (Element-wise 1), and element-wise outer product (Element-wise 2), as shown in Figure 2 right bottom. As shown in Figure 7, we depict a detailed breakdown of the memory read/write ratio with different input sequence length for various operations during Mamba computation process. It shows that the read/write ratio of the three operations differs by more than three orders of magnitude. For linear operations with more input and less output, existing optimization methods primarily focus on tiling [17] and load the tiled input to on-chip buffer to maximize data sharing. However, due to the computational characteristic of element-wise operations with more output and less input, it is redundant to apply input sharing methods like tiling.

6.2 Motivation and Insights

The computational characteristics of linear operations require reduction and input sharing, leading to a higher read/write ratio, which is suitable for existing input data sharing methods during each operation processing. By reviewing Figure 3, we find that the element-wise operations are closely spaced within the SSM computation process. And during the SSM process, the outputs of element-wise operations such as ΔA , ΔB , and h are accessed repeatedly. Storing these output of element-wise operations on the on-chip buffer can significantly reduce the overhead of memory access for the next operation. Therefore, our key insight is to **adjust**

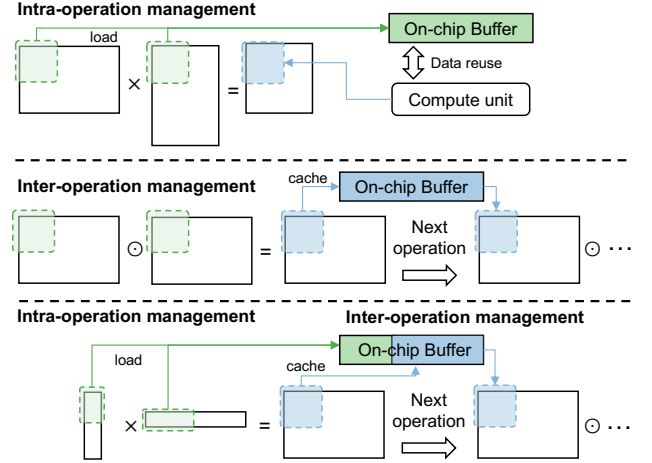


Figure 8: MARCA buffer management strategy for different operations.

different buffer management strategies for different operation types to maximize the data sharing with intra-operation and inter-operation.

6.3 Approach

Our operation-wise buffer management (BM) strategies contain intra-operation and inter-operation methods as shown in Figure 8. Intra-operation buffer management (Intra-BM) is used to discover and manage data sharing within individual operations, while inter-operation buffer management (Inter-BM) is used to discover and manage data sharing between operations.

Intra-operation Management. For linear operations, the whole on-chip buffers are configured as read buffers. The memory access handler loads linear inputs from global memory to fill the buffer. The computing unit then sequentially reads the required data from the buffer, performs calculations, and writes the computed results back to global memory. For element-wise 2 operation, even though the read/write ratio is relatively low, it can be effectively regarded as a matrix multiplication with two reduction dimensions of size 1, hence requiring data sharing among inputs. Therefore, we reserve a small fraction region of the on-chip buffer to store them.

Inter-operation Management. For element-wise 1, there is no data reuse in the computation of input data. The basic approach involves reading from global memory and directly writing back after computation. For individual computations, data reuse optimization like tiling is not feasible. For element-wise 1 and 2 operations, to maximize buffer utilization, we primarily optimize data sharing among adjacent element-wise operations. As shown in equation 2, the update of hidden state h is obtained by adding the product of ΔA and h with the product of ΔB and x . Therefore, during the continuous updating process of state h , h needs to be read and written L times repeatedly. Additionally, the corresponding ΔA , ΔB , and x need to be read L times repeatedly. Hence, for element-wise operations concentrated in the SSM process, we cache the above immediate result in the buffer.

Through an operation-wise buffer management strategy, our method maximizes the reduction in memory access and minimizes idle computational resources, thereby accelerating the Mamba computation process.

7 EXPERIMENTAL RESULTS

7.1 Experimental Setup

Methodology. The performance and energy of MARCA are measured by using the following tools.

Architecture Simulator. We design and implement a cycle-accurate simulator to measure execution time in number of cycles. This simulator models the microarchitectural behaviors of each module, which is integrated with Ramulator 2.0 [22] to simulate the behaviors of memory accesses to High Bandwidth Memory (HBM).

CAD Tools. We implement and synthesize our design in Verilog to measure area, power, and critical path delay (in cycles) for each module. We use the Synopsys Design Compiler with the TSMC 28 nm standard VT library for the synthesis, and estimate the power using Synopsys PrimeTime PX. The slowest module has a critical path delay of 0.9 ns including the setup and hold time, putting the MARCA comfortably at 1 GHz clock frequency.

Memory Measurements. The area, power, and access latency of the on-chip scratchpad memory are estimated using Cacti 7.0 [28]. Since Cacti only supports down to 32 nm technologies, we apply four different scaling factors to convert them to 28 nm technology as shown in [39]. The energy of HBM 1.0 is estimated with 7 pJ/bit as in [31].

Benchmark LLM Datasets. We conduct comprehensive experiments on the Mamba, which are owing to critical and efficient influence in recent model advancements. We depict Mamba models with different size and hyperparameters as shown in Table 1. We focus on two primary metrics: perplexity and zero-shot performance. The perplexity is evaluated by the WikiText [25] and Lambada [32] benchmarks. The zero-shot performance is assessed across four zero-shot benchmarks, namely Piqa [3], HellaSwag [45], WinoGrande [37], and Arc-easy [6].

Baseline Platform. To compare the performance and energy consumption of MARCA with state-of-the-art works, we evaluate Mamba model on a Linux workstation equipped with one Intel Xeon 8358P CPU [15] and a 252 GB DDR4 memory and one NVIDIA Tesla A100 GPU [30], denoted as Mamba-CPU and Mamba-GPU, respectively. Table 2 lists the system configurations for above implementations.

Table 1: Hyperparameters of Models in Mamba Family

Hyperparameters	130M	370M	790M	1.4B	2.8B
Layers	24	48	48	48	64
Hidden Size	768	1024	1536	2048	2560

7.2 Accuracy Evaluations

Table 3 illustrates the perplexity and zero-shot performance of the approximation algorithm on Mamba families, compared to the metrics computed using the original approach, along with the accuracy of the original fast exp algorithm applied to exponential and SiLU computations. The experimental results indicate that benefiting from the bias introduced based on the data distribution, our improved algorithm outperforms the fast exp algorithm across all sizes of Mamba. The average accuracy improvement ranges from 0.19% to 0.44%. The difference in accuracy compared to the original algorithm does not exceed 0.29%, indicating minimal loss in

Table 2: System Configuration.

	Mamba-CPU	Mamba-GPU	MARCA
Compute Unit	2.6GHz @ 32 Cores	1.4GHz @ 8192+512 Cores	1GHz @ 32 RCUs (each with 16×16 RPEs)
On-chip Memory	48MB	40MB	24MB
Off-chip Memory	136.5GB/s DDR4	2039GB/s HBM2e	256GB/s HBM1.0

Note: GPU’s on-chip memory includes the register files, and L1 and L2 caches. Mamba-GPU includes 8192 CUDA Cores and 512 Tensor Cores (each with 256 cores).

Table 3: Perplexity and Accuracy under varied approximation algorithms

Method	Perplexity (↓)	Accuracy (↑)		
	Wikitext/Lambada	Piqa/Wino./Arc-E/Hella.	Avg.(↑)	
Mamba-130M	26.25/16.04	63.17/52.33/42.09/35.23	48.21	
fast_exp	49.61/300.56	63.82/50.75/41.33/34.97	47.71	
Our_exp	27.36/19.49	63.22/51.62/41.84/35.01	47.92	
Our_silu	28.58/18.95	63.60/51.54/41.54/35.63	48.08	
Ours	29.69/18.50	63.93/51.54/40.87/35.46	47.95	
Mamba-370M	18.25/8.14	68.28/55.41/48.15/46.46	54.58	
fast_exp	29.46/136.70	68.72/55.25/47.81/45.36	54.28	
Our_exp	18.77/7.86	68.72/55.49/47.52/45.99	54.43	
Our_silu	20.07/8.62	68.55/55.88/47.14/47.69	54.82	
Ours	20.65/8.29	69.15/55.41/46.93/47.25	54.69	
Mamba-790M	15.06/6.01	72.58/55.64/53.83/55.05	59.28	
fast_exp	23.45/92.31	72.31/55.80/54.59/54.22	59.23	
Our_exp	15.39/6.60	72.47/56.59/54.50/54.48	59.51	
Our_silu	16.10/6.42	71.22/58.25/52.57/55.63	59.42	
Ours	16.64/6.53	72.47/57.70/52.69/55.61	59.62	
Mamba-1.4B	13.57/5.04	73.88/61.17/61.15/59.14	63.83	
fast_exp	20.66/58.84	73.45/60.77/60.23/58.29	63.18	
Our_exp	13.83/5.45	73.83/60.62/61.28/58.76	63.62	
Our_silu	14.71/5.92	73.88/59.75/59.09/59.28	63.00	
Ours	15.21/6.06	73.99/60.22/58.67/59.07	62.99	
Mamba-2.8B	11.76/4.23	75.79/63.38/64.27/66.17	67.40	
fast_exp	17.51/37.66	75.79/63.54/64.35/65.14	67.20	
Our_exp	11.98/4.52	75.57/63.38/65.19/65.40	67.39	
Our_silu	12.72/5.86	75.03/63.22/61.99/65.67	66.48	
Ours	13.12/6.08	75.63/64.25/60.56/65.91	66.59	

precision. After incorporating the piecewise SiLU into our complete algorithm, the maximum accuracy loss is only 0.84%. While employing the fast exp algorithm in SiLU yields higher accuracy, it introduces divide unit into PE unit, resulting in significant area overhead.

7.3 Hardware Evaluations

We compare our work with Mamba-CPU and Mamba-GPU in terms of speedup and energy consumption. Finally, the area and power of our design is presented.

Speedup. Figure 9 top depicts that MARCA achieves up to 463.32×/11.66× speedup and average 194.26×/4.93× speedup compared with Mamba-CPU and Mamba-GPU, respectively. The performance improvement comes from the reconfigurable architecture, and the intra-operation and inter-operation buffer management

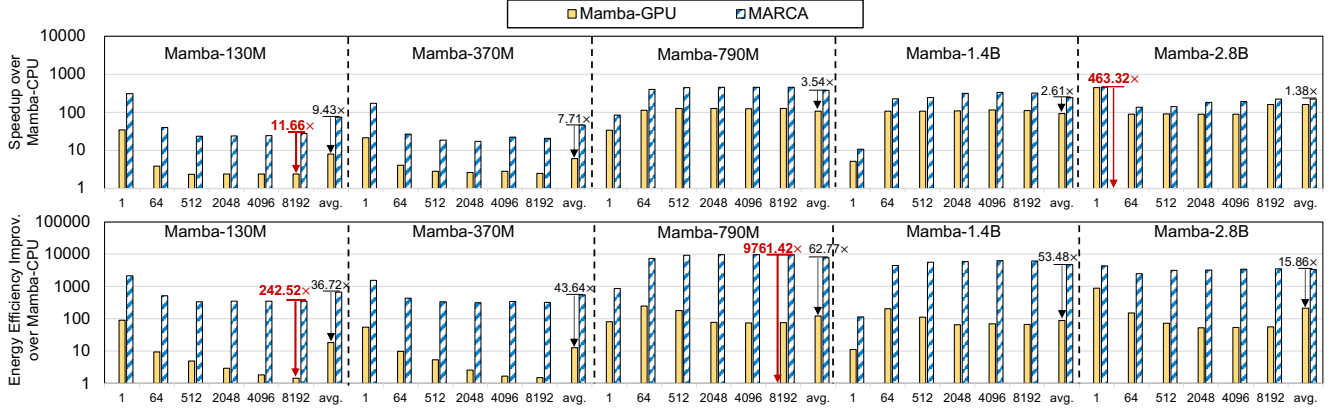


Figure 9: Comparison of speedup and energy efficiency improvement to Mamba-CPU and Mamba-GPU on Mamba models with different input sequence length.

strategy. First, the reconfigurable computing unit with several reconfigurable processing element arrays accelerate the computations. Second, the intra-operation input data sharing and inter-operation data sharing methods reduce large redundant data between global memory and on-chip memory.

Energy Efficiency. As Figure 9 bottom shows, MARCA improves up to $9761.42\times/242.52\times$ and average $3415.55\times/42.49\times$ energy efficiency compared to Mamba-CPU and Mamba-GPU, respectively. We consider the energy consumption of all platforms includes the off-chip memory.

Power and Area. The total power and area of MARCA are only 10.44 W and 221.88 mm^2 , respectively. For the on-chip buffer, we use eDRAM to reduce both the area and energy consumption. For the computation precision, we use 32-bit fixed point that is enough to maintain the accuracy of Mamba inference. Table 4 provides area and power breakdown. The on-chip buffer consumes most of power ($>60\%$) and area ($\sim 80\%$) to support more memory access and data sharing for element-wise operations. The compute engine consume 38.67% power and 20.57% area to perform the linear, element-wise, exponential, and SiLU computations. The others are small owing to the simple implementations of normalization, and instruction fetching and decoding.

Table 4: Layout Characteristics of MARCA

Component	Sub-module	Area ($\text{mm}^2/\%$)	Power ($\text{W}/\%$)
Inst. Processing	-	0.45/0.20%	0.045/0.43%
Norm. Unit	-	0.06/0.03%	0.003/0.03%
Compute Engine	RPEs	44.87/20.22%	3.92/37.55%
	Reduction Trees	0.47/0.21%	0.053/0.51%
	Control Unit	0.32/0.14%	0.064/0.61%
	-	45.66/20.57%	4.037/38.67%
On-chip Buffer	-	175.71/79.19%	6.35/60.87%
Total	-	221.88/100%	10.44/100%

7.4 Ablation Study

Speedup of RCU over Tensor Core. Figure 10 top left shows the speedup from $1.41\times$ to $11.95\times$ over Tensor Core-based architecture with different sequence length in Mamba.

Normalized area of RPE. In Figure 10 top right, we compare the normalized PE area overhead by supporting different nonlinear

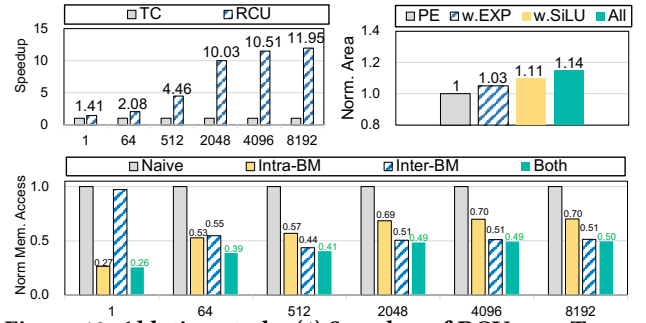


Figure 10: Ablation study: (1) Speedup of RCU over Tensor Core. (2) Normalized area of RPE. (3) Normalized memory access of intra-/inter-operation buffer management strategy.

functions. It shows that our reusable RPE only increases 14% area overhead.

Normalized memory access improvement. Figure 10 bottom reveals the normalized global memory access with out buffer management strategy. When the sequence is short, linear operations are dominant, the intra-BM reduces 73% memory access significantly. The inter-BM reduces 49% memory access with long sequence.

8 CONCLUSIONS

Our MARCA is the first proposed accelerator with reconfigurable architecture specifically tailored for Mamba computations. We propose a reduction alternative PE array architecture to support both linear and element-wise operations. Then, based on the reconfigurable PE, we decompose the nonlinear functions and reuse PE arrays reduce the area overhead. We also propose intra-operation and inter-operation buffer management strategy to maximize data reuse for two dominant operations. We conduct extensive experiments on Mamba model families with different model sizes. MARCA achieves up to $463.22\times/11.66\times$ speedup and up to $9761.42\times/242.52\times$ energy efficiency compared to Intel Xeon 8358P CPU and NVIDIA Tesla A100 GPU implementations, respectively.

9 ACKNOWLEDGMENTS

This work was supported by the National Natural Science Foundation of China (No. 62104128, U21B2031), Beijing Douyin Information Service Co., Ltd.

REFERENCES

- [1] Alan H Barr. 1991. The Einstein summation notation. *An Introduction to Physically Based Modeling (Course Notes 19)*, pages E 1 (1991), 57.
- [2] Ali Behrouz and Farnoosh Hashemi. 2024. Graph Mamba: Towards Learning on Graphs with State Space Models. *arXiv preprint arXiv:2402.08678* (2024).
- [3] Yonatan Bisk, Rowan Zellers, et al. 2020. Piqa: Reasoning about physical commonsense in natural language. In *AAAI*.
- [4] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. 2021. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258* (2021).
- [5] Jack Choquette, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. 2021. Nvidia ai100 tensor core gpu: Performance and innovation. *IEEE Micro* 41, 2 (2021), 29–35.
- [6] Peter Clark, Isaac Cowhey, et al. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457* (2018).
- [7] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xi-aohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).
- [8] Stefan Elfving, Eiji Uchibe, and Kenji Doya. 2018. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural networks* 107 (2018), 3–11.
- [9] Wade H Foy. 1976. Position-location solutions by Taylor-series estimation. *IEEE transactions on aerospace and electronic systems* 2 (1976), 187–194.
- [10] Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752* (2023).
- [11] Albert Gu, Karan Goel, Ankrit Gupta, and Christopher Ré. 2022. On the parameterization and initialization of diagonal state space models. *Advances in Neural Information Processing Systems* 35 (2022), 35971–35983.
- [12] Albert Gu, Karan Goel, and Christopher Ré. 2021. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396* (2021).
- [13] Albert Gu, Isys Johnson, Karan Goel, Khaled Saab, Tri Dao, Atri Rudra, and Christopher Ré. 2021. Combining recurrent, convolutional, and continuous-time models with linear state space layers. *Advances in neural information processing systems* 34 (2021), 572–585.
- [14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [15] INTEL. 2024. Intel Xeon Platinum 8358P Processor. <https://www.intel.com/content/www/us/en/products/sku/212308/intel-xeon-platinum-8358p-processor-48m-cache-2-60-ghz/specifications.html>.
- [16] Lincan Li, Hanchen Wang, Wenjie Zhang, and Adelle Coster. 2024. STG-Mamba: Spatial-Temporal Graph Learning via Selective State Space Model. *arXiv preprint arXiv:2403.12418* (2024).
- [17] Xiuhong Li, Yun Liang, Shengen Yan, Liancheng Jia, and Yinghan Li. 2019. A coordinated tiling and batching framework for efficient GEMM on GPUs. In *Proceedings of the 24th symposium on principles and practice of parallel programming*. 229–241.
- [18] Dingkan Liang, Xin Zhou, Xinyu Wang, Xingkui Zhu, Wei Xu, Zhikang Zou, Xiaoqing Ye, and Xiang Bai. 2024. PointMamba: A Simple State Space Model for Point Cloud Analysis. *arXiv preprint arXiv:2402.10739* (2024).
- [19] Jun Liu, Guohao Dai, Hao Xia, Lidong Guo, Xiangsheng Shi, Jiaming Xu, Huazhong Yang, and Yu Wang. 2023. TSTC: Two-level Sparsity Tensor Core Enabling both Algorithm Flexibility and Hardware Efficiency. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [20] Jiuming Liu, Ruiji Yu, Yian Wang, Yu Zheng, Tianchen Deng, Weicai Ye, and Hesheng Wang. 2024. Point mamba: A novel point cloud backbone based on state space model with octree-based ordering strategy. *arXiv preprint arXiv:2403.06467* (2024).
- [21] Siyuan Lu, Meiqi Wang, Shuang Liang, Jun Lin, and Zhongfeng Wang. 2020. Hardware accelerator for multi-head attention and position-wise feed-forward in the transformer. In *2020 IEEE 33rd International System-on-Chip Conference (SOCC)*. IEEE, 84–89.
- [22] Haocong Luo, Yahya Can Tu, F Nisa Bostanci, Ataberk Olgun, A Giray Ya, Onur Mutlu, et al. 2023. Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator. *IEEE Computer Architecture Letters* (2023).
- [23] Mishaim Malik, Muhammad Kamran Malik, Khawar Mehmood, and Imran Makhdoom. 2021. Automatic speech recognition: a survey. *Multimedia Tools and Applications* 80 (2021), 9411–9457.
- [24] Stefano Markidis, Steven Wei Der Chien, Erwin Laure, Ivy Bo Peng, and Jeffrey S Vetter. 2018. Nvidia tensor core programmability, performance & precision. In *2018 IEEE international parallel and distributed processing symposium workshops (IPDPSW)*. IEEE, 522–531.
- [25] Stephen Merity, Caiming Xiong, et al. 2016. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843* (2016).
- [26] AI Meta. 2023. Introducing LLaMA: A foundational, 65-billion-parameter large language model. *Meta AI* (2023).
- [27] Erxue Min, Runfa Chen, Yatao Bian, Tingyang Xu, Kangfei Zhao, Wenbing Huang, Peilin Zhao, Junzhou Huang, Sophia Ananiadou, and Yu Rong. 2022. Trans-former for graphs: An overview from architecture perspective. *arXiv preprint arXiv:2202.08455* (2022).
- [28] Naveen Muralimanohar, Rajeev Balasubramanian, and Norman P Jouppi. 2009. CACTI 6.0: A tool to model large caches. *HP laboratories* 27 (2009), 28.
- [29] Peter Nilsson, Ateeq Ur Rahman Shaik, Rakesh Gangarajiah, and Erik Hertz. 2014. Hardware implementation of the exponential function using Taylor series. In *2014 NORCHIP*. IEEE, 1–4.
- [30] NVIDIA. 2024. NVIDIA A100 Tensor Core GPU Architecture. <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf>.
- [31] Mike O'Connor. 2014. Highlights of the high-bandwidth memory (hbm) standard. In *Memory forum workshop*, Vol. 3.
- [32] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. 2016. The LAMBADA dataset: Word prediction requiring a broad discourse context. *arXiv preprint arXiv:1606.06031* (2016).
- [33] Johannes Partzsch, Sebastian Höppner, Matthias Eberlein, Rene Schüffny, Christian Mayr, David R Lester, and Steve Furber. 2017. A fixed point exponential function accelerator for a neuromorphic many-core system. In *2017 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 1–4.
- [34] Badri N Patro and Vijay S Agneeswaran. 2024. SiMBA: Simplified Mamba-Based Architecture for Vision and Multivariate Time series. *arXiv preprint arXiv:2403.15360* (2024).
- [35] William Peebles and Saining Xie. 2023. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 4195–4205.
- [36] Enrico Reggiani, Renzo Andri, and Lukas Cavigelli. 2023. Flex-sfu: Accelerating dnn activation functions by non-uniform piecewise approximation. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [37] Keisuke Sakaguchi, Ronan Le Bras, et al. 2021. Winogrande: An adversarial winograd schema challenge at scale. *Commun. ACM* (2021).
- [38] Nicol N Schraudolph. 1999. A fast, compact approximation of the exponential function. *Neural Computation* 11, 4 (1999), 853–862.
- [39] Aaron Stillmaker and Bevan Baas. 2017. Scaling equations for the accurate prediction of CMOS device performance from 180 nm to 7 nm. *Integration* 58 (2017), 74–81.
- [40] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [41] Chloe Wang, Oleksii Tsepa, Jun Ma, and Bo Wang. 2024. Graph-mamba: Towards long-range graph sequence modeling with selective state spaces. *arXiv preprint arXiv:2402.00789* (2024).
- [42] Teng Wang, Lei Gong, Chao Wang, Yang Yang, Yingxue Gao, Xuehai Zhou, and Huaping Chen. 2022. Via: A novel vision-transformer accelerator based on fpga. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 11 (2022), 4088–4099.
- [43] Jan C Willems. 1986. From time series to linear system—Part I. Finite dimensional linear time invariant systems. *Automatica* 22, 5 (1986), 561–580.
- [44] Yubiao Yue and Zhenzhang Li. 2024. Medmamba: Vision mamba for medical image classification. *arXiv preprint arXiv:2403.03849* (2024).
- [45] Rowan Zellers, Ari Holtzman, et al. 2019. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830* (2019).
- [46] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068* (2022).
- [47] Tao Zhang, Xiangtai Li, Haobo Yuan, Shunping Ji, and Shuicheng Yan. 2024. Point Cloud Mamba: Point Cloud Learning via State Space Model. *arXiv preprint arXiv:2403.00762* (2024).
- [48] Lianghui Zhu, Bencheng Liao, Qian Zhang, Xinlong Wang, Wenyu Liu, and Xinggang Wang. 2024. Vision mamba: Efficient visual representation learning with bidirectional state space model. *arXiv preprint arXiv:2401.09417* (2024).